

Data lake performance

Limiting data exchanges

Lesson objectives

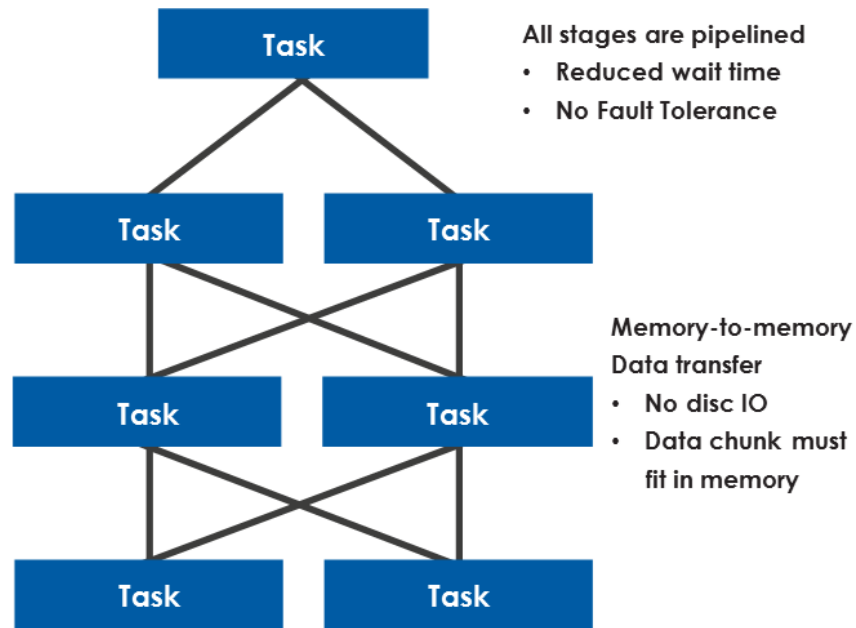
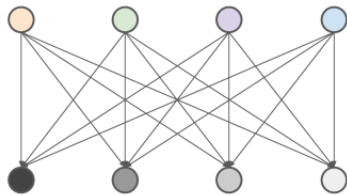
Data lake performance: Limiting
data exchanges

1. Describe the role that projection and filtering play on queries and the SQL commands needed to execute them.
2. Describe how data exchanges work and the impact they have on performance.

Project & filter early-and-often

Limit the amount of data transferred between query stages

- Operations that work independently on each row are pipelined together into a single task.
- Some operations require the entire dataset to be considered and require a new stage (ex: GROUP BY, JOIN, and ORDER BY).
 - Data is shuffled from many:many
- Referencing only columns you need (Projecting) and appropriate WHERE clauses (Filtering) limits:
 - Initial data read (especially on columnar files)
 - Amount of data transferred between stages
- These exchanges can get
 - very busy
 - expensive



Projection & filtering in action

Consider the query to the right.

- The `sales` table has 20 columns and is 1TB in size.
- We are only asking for 4 of those 20 columns.
- We are using a `WHERE` clause to filter down to 10% of the rows in the table.

Result: Although the table is large, we are only processing around 2% of the data after projecting and filtering.

```
SELECT
```

```
    order_id,  
    customer_name,  
    sale_amount,  
    sale_date
```

```
FROM
```

```
    sales
```

```
WHERE
```

```
    sale_amount > 1000  
    AND sale_date >= '2024-01-01';
```

Lesson summary

Data lake performance: Limiting data exchanges

1. Projection limits the number of columns being queried. Filtering limits the number of rows being queried. Both help to ensure efficiencies across your cluster.
2. Some operations, like GROUP BY, JOIN, and ORDER BY, require data to be exchanged between one stage and another. This is resource intensive.

Data lake performance

File format options

Lesson objectives

Data lake performance: File format options

1. Explain the difference between row-oriented and columnar file formats.
2. Describe the comparative advantages and disadvantages of major file formats.
3. Explain the embedded statistics that are recorded inside ORC and Parquet file formats and how they are leveraged.

Multiple file formats to choose from

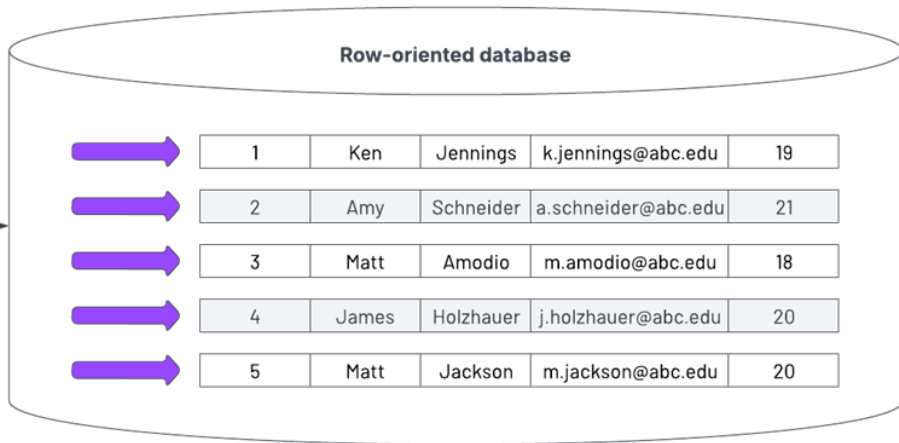
	CSV	JSON	Parquet	ORC	Avro
Columnar	X	X	✓	✓	X
Compressible	✓	✓	✓	✓	✓
Splittable	✓	✓	✓	✓	✓
Human-readable	✓	✓	X	X	X
Complex data structure	X	✓	✓	✓	✓
Schema evolution	X	X	✓	✓	✓

Row-oriented data storage

All data for a record is kept together and written in order of the fields. Subsequent records are appended to the end of the previous record.

File formats: delimited files (CSV), JSON, Avro

Student ID	First Name	Last Name	Email	Age
1	Ken	Jennings	k.jennings@abc.edu	19
2	Amy	Schneider	a.schneider@abc.edu	21
3	Matt	Amodio	m.amodio@abc.edu	18
4	James	Holzhauer	j.holzhauer@abc.edu	20
5	Matt	Jackson	m.jackson@abc.edu	20

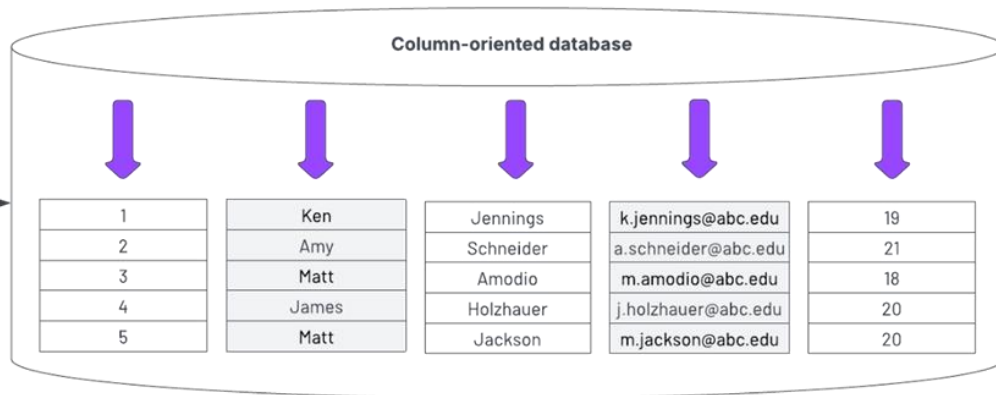


Columnar data storage

Data for a specific column (across all records) are stored independently from other columns. Low cardinality fields may employ a dictionary of unique values thereby shrinking the physical size of the data.

File formats: Apache Parquet, Apache ORC

Student ID	First Name	Last Name	Email	Age
1	Ken	Jennings	k.jennings@abc.edu	19
2	Amy	Schneider	a.schneider@abc.edu	21
3	Matt	Amodio	m.amodio@abc.edu	18
4	James	Holzhauer	j.holzhauer@abc.edu	20
5	Matt	Jackson	m.jackson@abc.edu	20



Columnar storage is ideal for analytics

Columnar storage dramatically improves performance for analytical queries by:

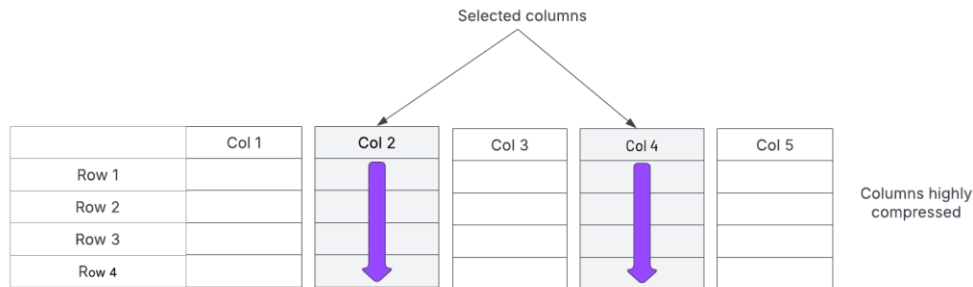
- ✓ Reducing disk I/O through selective column scans
- ✓ Improving compression efficiency
- ✓ Enabling faster aggregations and filtering with predicate pushdown
- ✓ Supporting parallel processing for large-scale analytics

For Starburst, using columnar formats like Parquet and ORC is a best practice for optimized query performance.

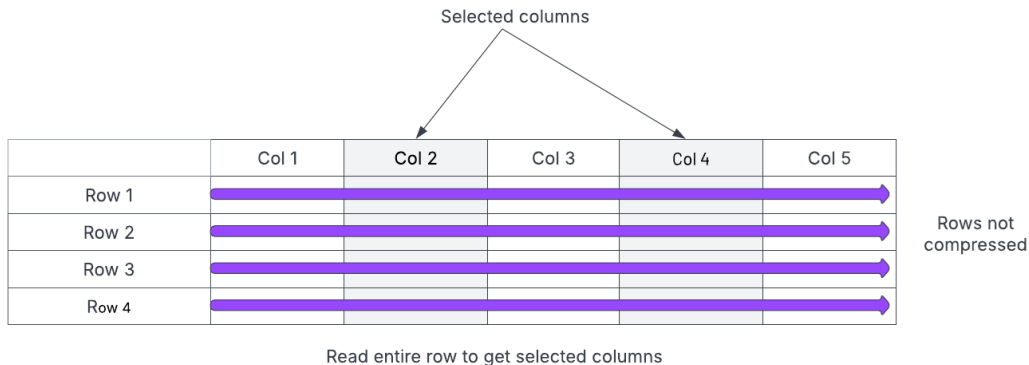


Row-oriented vs. columnar

Columnar file formats allow data to be pulled in from only the necessary columns.



Row-based file formats (like Text and Avro) pull in all data for a given row.



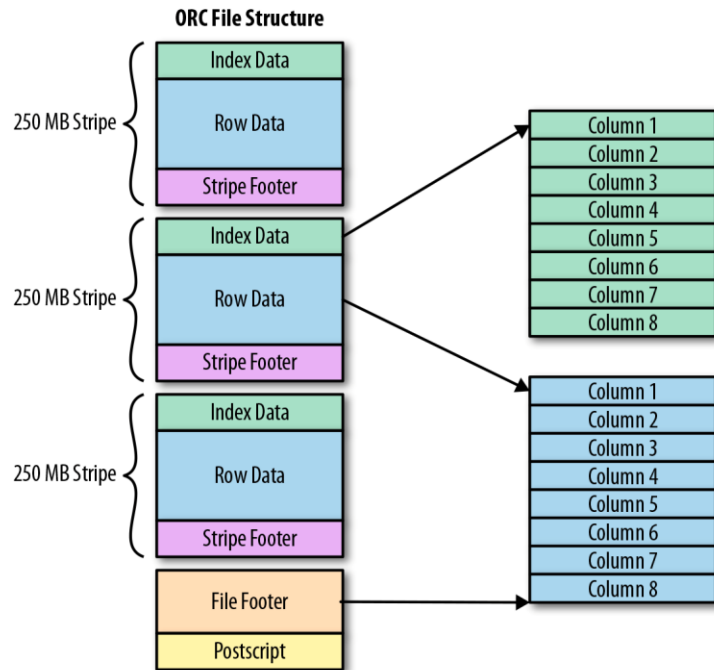
Multiple levels of statistics present

Internally sophisticated, but optimized for performance

ORC provides three level of indexes within each file:

- file level - statistics about the values in each column across the entire file
- stripe level - statistics about the values in each column for each stripe
- row level - statistics about the values in each column for each set of 10,000 rows within a stripe

The file and stripe level column statistics are in the file footer so that they are easy to access to determine if the rest of the file needs to be read at all. Row level indexes include both the column statistics for each row group and the position for seeking to the start of the row group.



Instructor demonstration

Exploring file formats (20 mins)



Lesson summary

Data lake performance: File format options

1. Row-oriented file formats organize data by rows. Each new row represents a new record. Examples include CSV, JSON, and AVRO.
2. Columnar file formats organize data by column. Examples include ORC and Parquet.
3. Columnar file formats hold a significant performance advantage, particularly for datasets with low cardinality columns, for queries that retrieve specific columns, and for analytical queries such as aggregations.
4. ORC and Parquet file formats embed additional metadata regarding statistics at the row, stripe/segment, and file levels.

Data lake performance

Small files problem

Lesson objectives

Data lake performance: Small files
problem

1. Explain the different factors that impact I/O performance costs in a cluster.
2. Describe the nature of the Small File Problem.
3. Demonstrate how file size impacts performance in real-world scenarios.

File size affects I/O costs

Consider this example

- 26 small files vs 2 large files, each containing the same payload.
- In the 26 small files case, lots of files will be opened (relatively expensive I/O overhead).
- In the 2 large files case, only a couple of files will be opened (relatively inexpensive I/O overhead).
- Object stores such as S3 can introduce high latency due to opening and reading many files.
- Multiple “compaction” options available.

Uncompacted data - 256MB



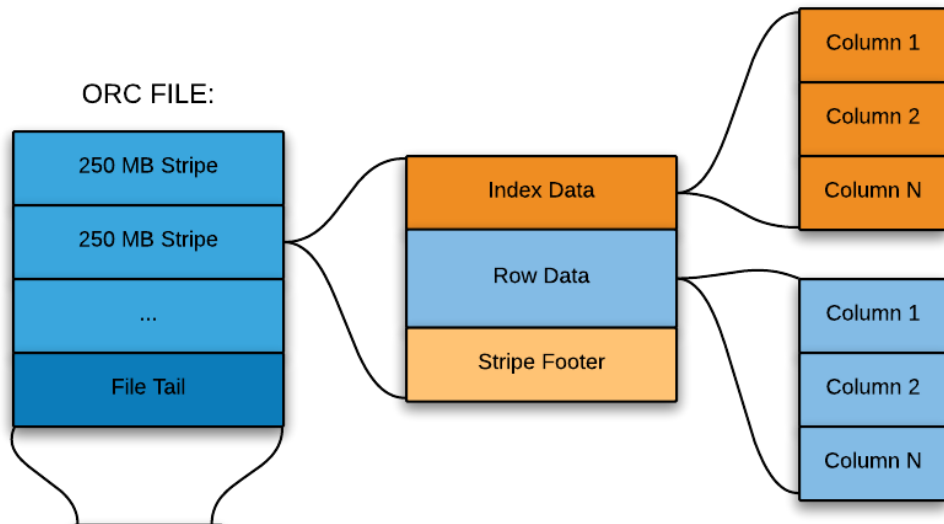
Compacted data - 256MB



Dimensions of the problem

There are two dimensions to the small file problem

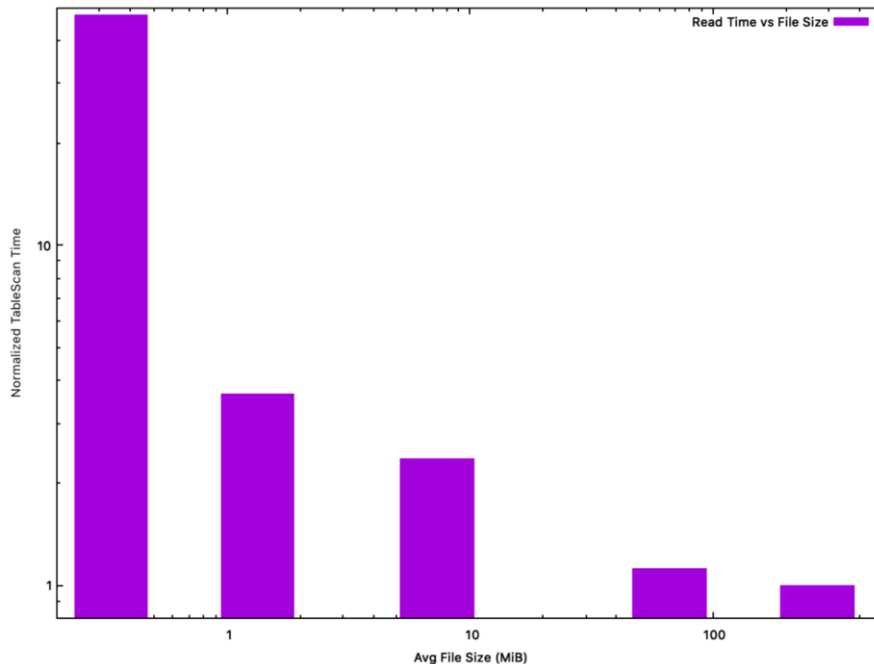
- The size of an individual file
- These internal segments are also known as row groups or stripes
- Very small internal segment sizes (i.e. 2MB) can manifest as poor performance with low CPU utilization



File size recommendations

Trino performs best when operating on individual file sizes > 10MB

- 10 MB is the lower threshold for avoiding the small files problem.
- The ideal size range is 64 MB to 512 MB.
- The general goal of larger files sizes (>10 MB) is to keep the ratio of data:metadata operations high (i.e. large:small).
- Multiple “compaction” options.



Instructor demonstration

File size performance impacts
(10 mins)

Hands-on exercises

Lab 1: Create tables with multiple
file formats (20 mins)

Lab 2: Using columnar file
formats and eliminating small
files (40 mins)

Lesson summary

Data lake performance: Small files problem

1. Smaller files represent a more significant performance overhead when compared to larger ones.
2. Because there are certain unavoidable costs to opening each file, it is not efficient to store data in many small files.
3. Starburst clusters perform best on files larger than 10 MB, and ideally in the 64 MB to 512 MB range.

Data lake performance

Partitioning & bucketing

Lesson objectives

Data lake performance: Partitioning
& bucketing

1. Explain the role that partitioning plays in data lakes.
2. Explain the role that bucketing plays in data lakes.
3. Explain how sorting and bloom filters provide additional sorting strategies.

Partitioning

Data lakes use partitioning as an alternative to indexing

- A partition is just a way to divide data into subdirectories that are designed around logical groupings.
- **Example:** Suppose you have 5 years of historical data.
 - Rather than dump all 5 years into a single folder, you might create a folder for each year, and then put each year's data into the appropriate folder.
 - When a query includes the partition key in the predicate or join, the query engine can avoid reading partitions that don't match.
- **Number of partitions not fixed:** Partitions are added as new data is added (IE: New year = new partition).

```
.../cat/sch/hist_tbl
|-- year=2020
|   |-- file1
|   |-- file2
|-- year=2021
|   |-- file3
|   |-- file4
|   |-- file5
|-- year=2022
|   |-- file6
|   |-- file7
```

Define partition columns at table creation

```
CREATE TABLE mycat.myschema.logs_daily_part_orc (  logs_daily_part_orc
```

```
    event_time      TIMESTAMP,  
    ip_address      VARCHAR(15),  
    app_name        VARCHAR(25),  
    process_id      SMALLINT,  
    log_type        VARCHAR(15),  
    log_level       VARCHAR(15),  
    message_id      VARCHAR(15),  
    message_details VARCHAR(555),  
    log_date        CHAR(10)  
  ) WITH (  
    format = 'ORC',  
    partitioned_by = ARRAY['log_date']  
  );
```

	event_time	timestamp(3)
	ip_address	varchar(15)
	app_name	varchar(25)
	process_id	smallint
	log_type	varchar(15)
	log_level	varchar(15)
	message_id	varchar(15)
	message_details	varchar(555)
	log_date	char(10)

Instructor demonstration

Partitioning performance
impacts (10 mins)

Bucketing

Another technique for dividing data into manageable chunks

- While partitions logically subdivide data into folders, buckets control the number of files within a folder.
- With bucketing, you choose a fixed number of buckets/files up front.
- Decide bucket count based on desired/ideal file size
- A consistent algorithm is used to determine the bucket assignment.
 - Bucketing keeps data that has the same key value in the same bucket.
 - This can improve performance, as matching data is always stored together in the same bucket.

BY ITSELF

```
.../cat/sch/hist_tbl
|-- bucketfile0
|-- bucketfile1
|-- bucketfile2
```

w/PARTITIONING

```
.../cat/sch/hist_tbl
|-- year=2020
|   |-- bucketfile0
|   |-- bucketfile1
|   |-- bucketfile2
|-- year=2021
|   |-- bucketfile0
|   |-- bucketfile1
|   |-- bucketfile2
|-- year=2022
|   |-- bucketfile0
|   |-- bucketfile1
|   |-- bucketfile2
```

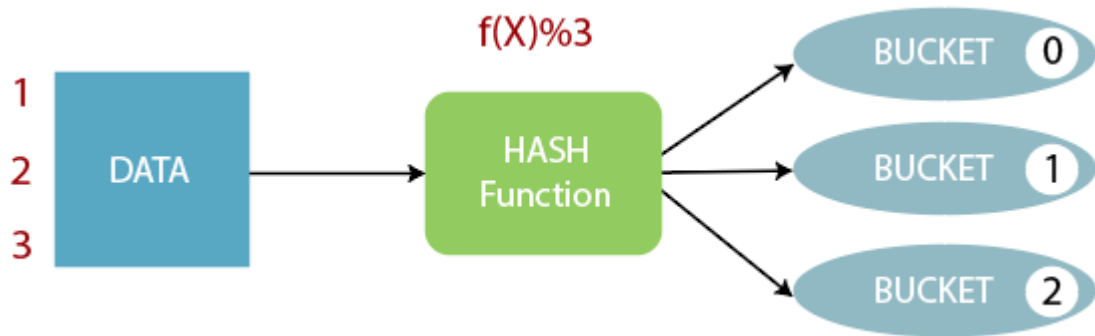
Define bucketing details at table creation

```
CREATE TABLE hive.web.page_views (  
    view_time timestamp,  
    user_id bigint,  
    page_url varchar,  
    ds date,  
    country varchar  
)  
WITH (  
    format = 'ORC',  
    partitioned_by = ARRAY['ds', 'country'],  
    bucketed_by = ARRAY['user_id'],  
    bucket_count = 50  
);
```

Hashing: The bucket assignment algorithm

A hash function is a way to map data of arbitrary size into a fixed number of buckets.

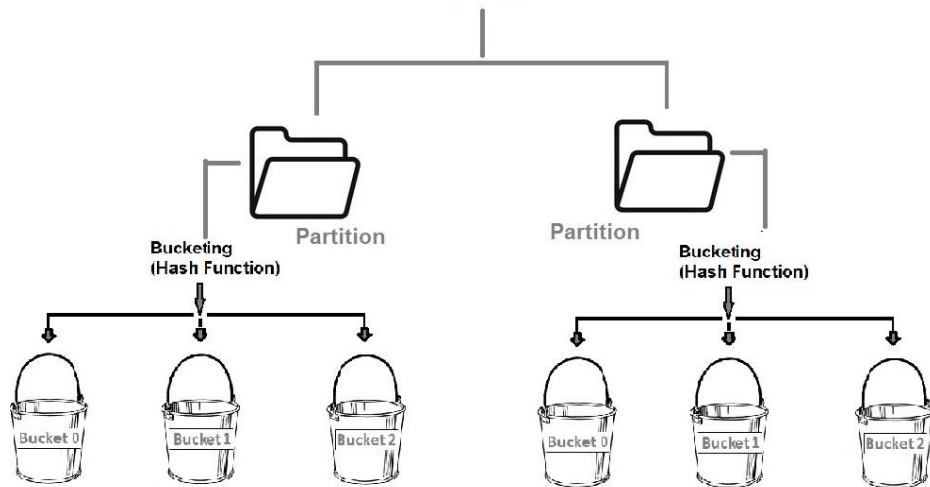
- An example of a simple hash function is “modulo,” which provides the remainder after dividing two values.
- **For example:** $4 \bmod 3 = 1$ because 4 divided by 3 leaves a remainder of 1.
- Hash functions are used to map values in joins to buckets in a hash table, or to map them to specific server nodes for processing.



Bucketing vs. partitioning

Each solves a different problem.

- Both are useful to limit the amount of data to read.
- **Partitions:**
 - Implemented using folder structures
 - Work best on low cardinality columns that have fairly uniform distribution
- **Buckets:**
 - Implemented by splitting files
 - A way to continue to subdivide data that may not make sense to partition, such as high cardinality and/or highly skewed columns
- BE CAREFUL OF CREATING SMALL FILES USING EITHER/BOTH OF THESE FEATURES



Additional strategy: Sorting

Total order sorting for a table or partition

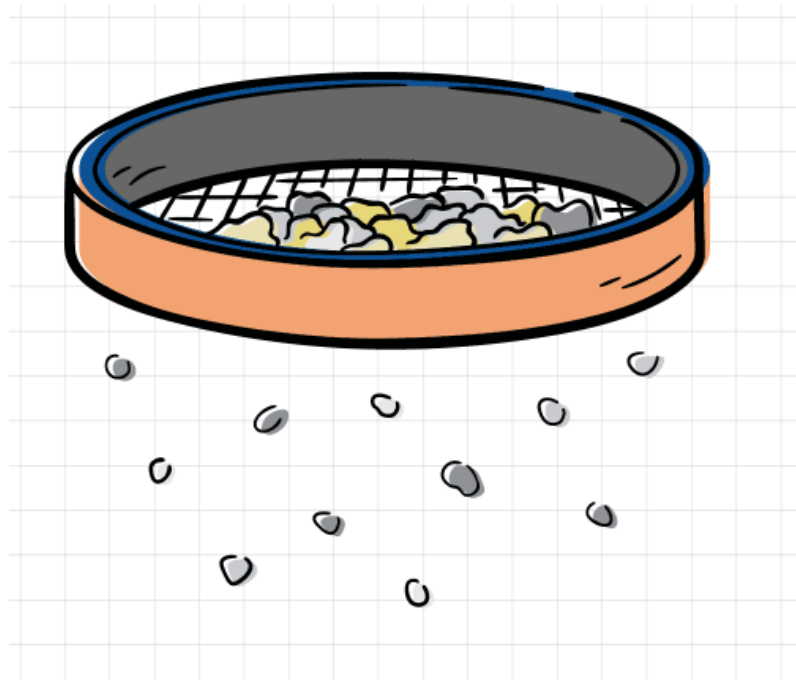
- Sorting all of the files for a specific table, or partition, on heavily queried column(s) allows for index-like behavior.
- Sorting uses the statistics associated with each file, including min & max values, and checks to see if a particular WHERE selection is either:
 - Potentially in the file
 - Definitely not in the file
- Like partitioning & bucketing, this is another way to intelligently limit the amount of data it has to read

select * from tbl where id between 555 and 777;		
Unsorted		Sorted
Field: id		Field: id
min 101, max 999	file A	min 101, max 122
min 150, max 999	file B	min 123, max 244
min 101, max 955	file C	min 245, max 490
min 110, max 988	file D	min 490, max 829
min 101, max 999	file E	min 830, max 999
	READ	
	SKIPPED	

Additional strategy: Bloom filters

Another way to walk away from a file

- A space-efficient, probabilistic data structure used to test if a specific column value could possibly be in a particular file.
- False positives are possible, but false negatives are not.
- Useful with high cardinality columns where the value to be searched is rarely present



Hands-on exercise

Exploring table partitioning &
bucketing (40 mins)

Lesson summary

Data lake performance: Partitioning & bucketing

1. Partitioning separates data into folders based on a low cardinality column's value and allows a WHERE clause to determine a subset of these folders to be read.
2. Using a hashing algorithm, bucketing ensures all data with the same value of a high cardinality column is persisted together in the same file thus allowing a WHERE clause to determine which file(s) to read.
3. Total order sorting across files allows the statistics associated with columnar formats to be utilized by the query engine to quickly stop reading a file if a particular value, or range of values, could not be present.
4. A bloom filter is a probabilistic technique that can allow the majority of files to not have to be read when querying for rare values in high cardinality columns.